

Swift Pay QA Review

“Scan to Pay” (QR) Feature — QA & Java Backend Capstone Review

Katy Great Adonai
Java Backend Developer (Capstone Submission)

August 15, 2026

Contents

1 Positive Test Case	2
2 Negative/Edge Test Case	3
3 Bug Report	4
4 Process Improvement Suggestion (Java Backend Perspective)	8

Reviewed against Feature Spec FS-P2M-v1.0 and the Figma UI flow text for both QR scenarios (with and without an embedded amount). Findings below are based on the UI copy and screen sequence as extracted from the design file; no Figma inspection or screenshots beyond the provided text were used.

1 Positive Test Case

Test Case ID	TC-STP-001
Title	Verify successful merchant payment via QR code without an embedded amount (Kofi's Cafe)
Preconditions	• User is logged into an active Swift Pay account • Wallet balance shows GHS 150.00 • Camera permission is granted • Merchant QR code for Kofi's Cafe (order: 1x Americano, single, iced, medium, full ice, GHC 15.00) does not carry an embedded amount
Steps	1. From the home screen, tap "Scan QR Code" 2. On the "Scan to Pay" screen, point the camera at the Kofi's Cafe QR code within the frame 3. Wait for "Processing QR Code" and then "Scan successful" 4. On the order/confirmation screen, review merchant name, order (Americano, single/iced/medium/full ice, GHC 15.00, x1) 5. Enter amount: 15 6. Tap "Confirm Payment" 7. Enter PIN: 1212 8. Wait for "Processing transaction"
Expected Results	• Merchant name "Kofi's Cafe" and order details are displayed correctly and match the QR code • Amount field accepts manual entry and updates the displayed total consistently across the confirmation and PIN screens • "Confirm Payment" only becomes active once a valid amount is entered • "Payment Successful!" screen displays: amount paid, merchant name ("Paid to Kofi's Cafe"), order, transaction ID, and date/time • Wallet balance is debited from GHS 150.00 to GHS 135.00 and reflected immediately • "Download Receipt" and "Back to Home" options are both available and functional

Note: A parallel positive case (TC-STP-001b) applies to the QR-with-embedded-amount

flow, where the amount (¢15.00) should be pre-populated and read-only, and the user proceeds directly from scan to "Confirm Payment" without a manual entry step.

2 Negative/Edge Test Case

Test Case ID	TC-STP-002
Scenario	Incorrect PIN entry: user reaches the PIN screen (either QR flow) and enters an incorrect PIN before retrying with the correct one.
Preconditions	• User has scanned a valid Kofi's Cafe QR code and reached the "Enter pin" screen • Correct PIN is known to be 1212
Steps	1. Enter an incorrect PIN (e.g., 1111) 2. Observe the screen response 3. Re-enter the correct PIN (1212) and confirm payment completes
Expected Result	The app displays an "Incorrect pin" state and keeps the user on the PIN entry/confirmation screen without deducting the wallet balance or generating a transaction ID. The user can then retry with the correct PIN and complete the payment normally. No retry limit, lockout behavior, or maximum-attempt messaging is shown anywhere in the provided flow — this gap is reported separately as BUG-STP-006 below.

Test Case ID	TC-STP-003
Scenario	Invalid QR code: user scans a QR code that the app cannot process.
Preconditions	User is on the "Scan to Pay" screen with camera permission granted
Steps	1. Point the camera at an invalid or unreadable QR code 2. Observe the app's response
Expected Result	The app displays "QR Code invalid. Scan Again" and keeps the user on (or returns them to) the scan screen. No navigation to a confirmation or payment screen should occur, and no charge should be possible from an unrecognized code.

Test Case ID	TC-STP-004
Scenario	Missing amount in the non-embedded QR flow: user attempts to confirm payment while the amount field is still at its default ₦0.00.
Preconditions	Kofi's Cafe QR code (non-embedded-amount flow) has been scanned and the confirmation screen is showing
Steps	1. Leave the amount field at ₦0.00 (do not enter a value) 2. Attempt to tap "Confirm Payment"
Expected Result	"Confirm Payment" should remain inactive/disabled, or tapping it should surface a validation message requiring a non-zero amount. <i>Note: the provided flow text does not show a distinct disabled or error state for this condition, so this is flagged for confirmation with the design/dev team rather than treated as a confirmed defect.</i>

3 Bug Report

Bug ID	BUG-STP-001
Title	Inconsistent currency symbol/format across screens
Description	The same currency is represented three different ways across the flow: the cedi sign "¢" (amount field, authorization screen), the abbreviation "GHC" (order price, e.g. "GHC 15.00"), and "GHS" (wallet balance, e.g. "GHS 150.00"). These should all refer to the same Ghanaian cedi.
Steps to Reproduce	Scan a Kofi's Cafe QR code and progress through the confirmation, order, balance, and payment screens, noting the currency label used on each.
Expected Result	A single, consistent currency symbol or code (e.g. "GHS" everywhere, or "¢" everywhere) is used across all screens in the flow.
Actual Result	Three different currency representations ("¢", "GHC", "GHS") appear across different screens within the same payment journey.
Severity / Priority	Minor / Medium — does not block a transaction but undermines user trust and looks unpolished in a financial app.

Bug ID	BUG-STP-002
Title	Malformed/leaked text on the fee line
Description	On the PIN authorization screen, the fee line renders as Fee: "GHS0.00", with literal quotation marks and no space between the currency code and the amount.
Steps to Reproduce	Scan any valid QR code, proceed to "Confirm Payment," and view the authorization/PIN screen.
Expected Result	Fee should read as clean, formatted text, e.g. Fee: GHS 0.00.
Actual Result	Literal quote characters appear around the value, indicating the string is not being formatted/templated correctly before being displayed.
Severity / Priority	Minor / Low — cosmetic, but suggests a templating bug that should be checked in case it affects other dynamic values.

Bug ID	BUG-STP-003
Title	Mismatched/leftover copy on the PIN entry screen
Description	The PIN entry screen displays: "Tax:0 from your account to MTN for your Request Service. Enter your pin to continue." This text references "MTN" and a "Request Service," neither of which applies to a QR payment to Kofi's Cafe.
Steps to Reproduce	Scan a Kofi's Cafe QR code, tap "Confirm Payment," and read the descriptive text above the PIN pad.
Expected Result	The descriptive text should reference the actual merchant and transaction type, e.g. "Pay GHS 15.00 from your account to Kofi's Cafe. Enter your PIN to continue."
Actual Result	The copy appears to be reused from an unrelated payment type (e.g., airtime/bill payment to a telco), and was not updated for the Scan to Pay / P2M flow.
Severity / Priority	Major / High — this is user-facing, incorrect, and could reasonably make a user question whether they are authorizing the right payment.

Bug ID	BUG-STP-004
Title	Embedded amount field is visually identical to the editable amount field
Description	Per AC-09, when a QR code carries an embedded amount, the user should not be able to edit it. In the provided flow, the amount is pre-filled (¢15.00) but is displayed in what appears to be the same input-style field used in the non-embedded flow (where the field starts at ¢0.00 and is user-editable), with no visible read-only styling, lock icon, or other indicator that it is fixed.
Steps to Reproduce	Compare the confirmation screen amount field in the "with embedded amount" flow to the same field in the "without embedded amount" flow.
Expected Result	The embedded-amount field should be visually distinguishable as non-editable (e.g., greyed out, locked icon, static text rather than an input box) so users don't attempt to tap or edit it.
Actual Result	Both fields appear to use the same input-box styling, with the only difference being the pre-filled value. Whether editing is actually blocked cannot be confirmed from the provided text, but the lack of visual distinction is itself a usability concern tied directly to AC-09.

Severity / Priority	Minor / Medium — doesn't necessarily break the AC functionally, but risks user confusion and support tickets ("why can't I fix my order amount?").
----------------------------	--

Bug ID	BUG-STP-005
Title	Repeated typo in scan-screen guidance text
Description	The scan guidance text reads "Keep the QR code within the frame for best results. The app will scan automatically." — "automatically" is misspelled with an extra "l," and this typo repeats across multiple instances of the same screen in the provided flow text.
Steps to Reproduce	Open "Scan to Pay" and read the on-screen guidance text.
Expected Result	"The app will scan automatically."
Actual Result	"The app will scan automaticallyl."
Severity / Priority	Trivial / Low — cosmetic copy fix, but easy to catch and easy to miss in a content review pass.

Bug ID	BUG-STP-006
Title	No PIN retry limit or lockout behavior defined or shown
Description	The flow shows a single "Incorrect pin" state (TC-STP-002) but no maximum-attempt counter, lockout screen, or step-up authentication (e.g., OTP) after repeated failures. For a financial transaction PIN, this is a security gap as much as a UX one.
Steps to Reproduce	Reach the "Enter pin" screen and enter an incorrect PIN multiple times in a row; observe that the same "Incorrect pin" state is the only behavior shown, with no indication of remaining attempts or an eventual lockout.
Expected Result	After a defined number of consecutive failed attempts (e.g., 3), the app should lock the transaction PIN temporarily and/or require step-up authentication, with a clear message explaining the lockout and cooldown period.
Actual Result	No retry limit, attempt counter, or lockout state is present anywhere in the provided flow — PIN entry appears to be retryable indefinitely.
Severity / Priority	Major / High — an unlimited-attempt PIN on a payment flow is a brute-force risk, not just a missing polish item.

4 Process Improvement Suggestion (Java Backend Perspective)

Several of the issues above – the fee line rendering with stray quote characters, the mismatched “MTN / Request Service” copy on the PIN screen, and the currency label drifting between “¢,” “GHC,” and “GHS” – are all symptoms of the same root cause from a backend point of view: the values and message templates being sent to the client for a given transaction type were never validated against that transaction type before the UI was built against them.

The one concrete action I’d recommend: **introduce automated backend contract and business-rule tests for the QR payment endpoint(s), covering both the embedded-amount and non-embedded-amount cases, before the UI/integration testing phase begins.** Concretely, this means writing JUnit-based integration tests (e.g., with Spring Boot Test and a test database or WireMock for downstream services) that assert, for each payment type:

- the response payload always carries a single, canonical currency field (e.g., “currency”: “GHS”) rather than allowing free-text or per-endpoint variation in how currency is labeled;
- message/template fields returned to the client (fee descriptions, authorization prompts) are generated from the correct transaction-type template – so a P2M QR payment can never receive the string set built for a telco/airtime “Request Service” flow;
- the embedded-amount field in the QR payload is marked immutable at the API level (not just conventionally left alone by the client), so the backend rejects any client-submitted amount that doesn’t match the QR’s embedded value;
- PIN validation responses distinguish clearly between “incorrect PIN” and “locked out,” with the retry-count and lockout logic enforced server-side, not left for the UI to infer.

Because these are backend contract and business-rule issues, not visual ones, they can be tested completely before a single UI screen is wired up. Writing these as automated tests during API development means a bad template string, a currency mismatch, or a missing amount-immutability check fails a build in CI – rather than surfacing three weeks later during UI/QA testing (as it effectively did here) or, worse, in production. That’s the Shift Left principle in practice for a backend engineer: push the business-rule validation as close to the code that creates the data as possible, so QA at the UI layer is confirming presentation, not discovering that the data underneath was wrong all along.